



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Lista 06 - Filas

Professora: *Marta Noronha*

Disciplina: *Algoritmos e Estrutura de Dados II*

Data de entrega: 31/05/2026

Requisitos

1. Todos os programas deverão ser desenvolvidos na linguagem de programação Java.
2. Cópias, se existirem, serão encaminhadas ao colegiado de coordenação didática do curso.
3. Fique atento ao charset dos arquivos de entrada e saída. Recomenda-se a utilização dos métodos da classe `MyIO.java` para leitura de dados do teclado. É necessário definir o charset a ser utilizado antes de começar a leitura de dados do teclado, da seguinte forma:
`MyIO.setCharset("ISO-8859-1").`
4. As saídas esperadas, cadastradas no **VERDE** pelo professor, foram geradas empregando-se:
`System.out.println().`
5. Os arquivos `pub.in` e `pub.out` estarão disponíveis no **VERDE** se forem necessários para a execução dessa atividade.
6. Em cada submissão, enviar apenas um arquivo (.java). A regra será necessária para a submissão de exercícios no **VERDE** e no identificador de plágios utilizado na disciplina.
7. A resolução (código) de cada exercício deverá ser submetida ao **VERDE**.
8. A execução do código submetido será realizada automaticamente pelo **VERDE**, mas o código será analisado e validado pelo professor.

Criação da classe Jogo

O conjunto de dados utilizado neste exercício foi disponibilizado em Kaggle.

Implemente uma classe `Jogo` com os seguintes atributos declarados com modo de acesso privado:

- `rank` (int): classificação (posição) do jogo em relação aos demais.
- `nome do jogo` (String): o nome do jogo.
- `plataforma` (String): Onde o jogo pode ser executado (Wii, DS, entre outros).

- ano (int): ano de lançamento.
- gênero (String): esportes, corrida, entre outros.
- editora (String): empresa que disponibiliza o jogo (produtora).
- NA_Vendas (double): Vendas do jogo na América do Norte.
- EU_Vendas (double): Vendas do jogo na Europa.
- JP_Vendas (double): Vendas do jogo no Japão.
- Outras_Vendas (double): Vendas do jogo em outros lugares.
- Vendas_Global (double): Vendas global do jogo.

A classe também deve conter, obrigatoriamente, ao menos, dois construtores (1. *padrão (default)*; 2. (*nome do jogo*)/*plataforma/ano*), e os métodos *gets*, *sets*, métodos *clone()*, *ler()*, *imprimir()* e *toString()*.

O método *clone()* deve retornar um objeto da mesma classe e contendo os mesmos valores de atributos do atual objeto analisado (**vide exemplo do método clone no slide "Unidade 0 - Nivelamento - Ponteiros (C/C++) e Referências (Java)"** postado na disciplina teórica). **Não é permitido usar a interface Cloneable para esta finalidade.**

O método *ler()* deve receber cada linha do arquivo como *parâmetro*. Cada linha possui os dados de atributos de cada objeto do tipo Jogo. O método deve armazenar os valores contidos em cada linha nos atributos de cada objeto que foi instanciado.

O método *toString()* deve ser implementado para permitir a impressão de um objeto da classe Jogo, sem necessidade de invocação do método *imprimir()*. Isso é feito pela sobrescrita do método *toString()*, um método implementado na super classe **Object**. Esse método retorna, por padrão, uma *String* contendo as informações do objeto, o qual deve ser implementado para apresentar as informações conforme o modelo indicado no fim deste documento. [Clique AQUI para ver o modelo.](#)

Em Java, a classe **Object** define os comportamentos mínimos que todos os objetos possuem. Um desses métodos é o **toString()**, cujo propósito é fornecer uma representação textual do objeto. Por padrão, a implementação de *toString()* na classe **Object** **retorna uma string** contendo o nome da classe seguido de um código hash, mas é comum que os desenvolvedores sobrescrevam esse método para fornecer uma representação mais significativa, geralmente exibindo os valores dos atributos do objeto.

Todas as classes em Java, inclusive a que contém o método *main*, herdam da classe **Object**. Dessa forma, ao sobrescrever o método *toString()*, o desenvolvedor redefine a forma como o objeto é convertido em string, permitindo uma representação customizada e mais informativa.

O método *imprimir()* é um método público sem retorno, implementado na classe Jogo, que imprime na saída padrão a representação textual de um objeto do tipo Jogo. Para isso, o método deve simplesmente chamar o método de impressão informando, como parâmetro, o objeto por meio da palavra-chave *this*.

```
public void imprimir() {  
    System.out.println(this);  
}
```

Aqui, *println(Object)* recebe o próprio objeto (*this*), chama automaticamente o método sobrescrito *toString()* e exibe a string formatada conforme o modelo exigido.

Os métodos que não obedecem a forma como devem ser implementados, conforme descrição acima, serão penalizados ainda que o envio no Verde contabilize 100% de acerto. Isso será feito porque o desenvolvedor deve seguir atentamente as regras de implementação de um método, quando houver, considerando os parâmetros e o tipo de retorno que um método deve possuir quando estes são solicitados. Em caso de dúvidas, consulte a Professora.

Após a criação da classe, deve ser criado um mecanismo para processamento de uma entrada de dados. A entrada de dados é dividida em 2 partes:

- Parte 1: Armazenamento de informações contidos no arquivo jogos.txt em vetor
- Parte 2: Pesquisa de informações armazenadas no vetor criado na parte 1.

Exercício 1 - Fila circular implementada por vetor (1,5 PONTOS)

Parte 1 (Leitura de Arquivo): Armazenamento de informações contidos no arquivo jogos.txt em vetor

Seu programa deve ler um arquivo-texto chamado "jogos.txt" que, no VERDE, localiza-se na pasta "/tmp".

O aluno(a) deve preencher um vetor de objetos da classe Jogo com os dados dos diversos jogos informados no arquivo "jogos.txt". Considere que o vetor de jogos pode ser preenchido com até 1500 jogos, ainda que algumas posições fiquem vazias após a inserção dos jogos neste vetor.

NÃO DEVE SER USADO O ARRAYLIST PARA ARMAZENAR OS OBJETOS DESTA TAREFA. O USO DO ARRAYLIST PARA RESOLUÇÃO DESTA LISTA IMPLICA NO RECEBIMENTO DA NOTA ZERO.

Cada uma das linhas seguintes apresenta os dados de um jogo, separados pelo símbolo "(Use "\\|\" para fazer o split do vetor). Os dados possuem, em ordem, as seguintes informações:

- rank (int);
- nome do jogo (String);
- plataforma (String);
- ano (int);
- gênero (String);
- editora (String);
- NA_Vendas (double);
- EU_Vendas (double);
- JP_Vendas (double);
- Outras_Vendas (double);
- Vendas_Global (double);

A última linha do arquivo é vazia.

Parte 2: Pesquisa de informações armazenadas no vetor criado na Parte 1.

Após o processamento da primeira parte da entrada de dados, o programa deve processar a entrada contida no arquivo *pub.in*. Cada linha da segunda parte contém, em ordem, as seguintes informações:

- nome do jogo;
- ano;
- editora;
- plataforma.

A última linha do arquivo *pub.in*, usada para a parte 2 desta lista, contém a palavra FIM.

As entradas devem ser pesquisadas no vetor de jogos, criado na parte 1. Os registros encontrados na pesquisa devem ser armazenados em uma **FILA CIRCULAR, capaz de armazenar até 60 jogos simultaneamente**, que será utilizado na Parte 3 desta prática.

Os métodos de sua fila circular devem operar conforme descrito a seguir, respeitando-se parâmetros e tipos de retorno:

- Sua classe Fila deverá ter dois construtores;
- `void enfileirar (Jogo jogo)`: enfileira um objeto do tipo Jogo;
- `Jogo desenfileirar ()`: desenfileira e retorna o jogo da frente da fila;
- `void mostrar ()`: para todos os objetos do tipo Jogo presentes na fila, exibe a posição do objeto na fila seguida dos valores de seus atributos (observe o formato de cada linha da saída esperada);
- `obterSomaVendasGlobal()`: deve atualizar o total de vendas global a cada jogo incluído ou removido da fila circular. O resultado deve ser arredondado para ser retornado como um inteiro.

A execução desta parte do trabalho deve ser finalizada após a leitura da palavra FIM.

O tamanho da fila deve ser atualizado a cada operação de inserção ou remoção.

A cada operação de inserção na fila, deve ser impresso a soma atualizada das vendas globais.

Se, no momento de execução da operação enfileirar, a fila estiver cheia, antes de enfileirar um jogo será necessário desenfileirar outro.

O programa deve contabilizar a quantidade de jogos lidos na entrada padrão, neste caso *pub.in*, que **foram** encontrados no arquivo *jogos.txt*.

Ao final da segunda parte, após ler os registros que foram pesquisados no *pub.in*, deverão ser impressas as informações:

Quantidade de jogos encontrados: {total de jogos}

Tamanho da fila: {tamanho da fila}

(Obs: as chaves não devem ser impressas)

Parte 3: Enfileiramentos e desenfileiramentos na fila criada na Parte 2.

A primeira linha da segunda parte da entrada padrão (*pub.in*) apresenta um número inteiro n indicando a quantidade de jogos que serão enfileirados ou desenfileirados. Nas próximas n linhas, tem-se n comandos de enfileiramento ou desenfileiramento, que devem ser processados. Cada uma dessas linhas tem uma palavra de comando, conforme descrito a seguir:

- E: enfileirar;
- D: desenfileirar.

No caso dos comandos de enfileiramento, temos também uma string contendo as seguintes informações, que devem ser pesquisadas no vetor de jogos da Parte 1:

- nome do jogo;
- ano;
- editora;
- plataforma.

Após encontrar o registro no vetor de jogos, adicioná-lo à fila. **Se, no momento de execução da operação enfileirar, a fila estiver cheia, antes de enfileirar um jogo será necessário desenfileirar outro.**

A saída padrão será composta por:

- um número inteiro correspondente ao total de vendas global de todos os jogos contidos na fila, após cada enfileiramento.
- Uma linha para cada jogo desenfileirado, sendo que essa informação será constituída pela string “(D)” seguida dos atributos desse jogo.
- Ao final da execução, devem ser impressas as informações dos jogos armazenados na fila (observe o formato de cada linha da saída esperada).

Para o caso do desenfileiramento deve ser impresso:

(D) [Nome do jogo] [Plataforma] [Vendas global] {Rank}. {Gênero}. {Editora}. Mais vendido:{ País ou região mais vendido}.

Exemplo da saída após desenfileirar:

(D) [Pro Yaky? Spirits 5] [DS] [0.08] 11614. Adventure. Square Enix. Mais vendido: JP_Vendas.

Para o caso da impressão dos jogos que estão na fila, deve ser impresso:

[posição na fila] [Nome do jogo] [Plataforma] [Vendas global] {Rank}. {Gênero}. {Editora}. Mais vendido:{ País ou região mais vendido}.

Exemplo da saída:

[4][Skate 3] [Wii] [28.62] 9. Platform. Nintendo. Mais vendido: NA_Vendas.

Exercício 02 - Fila implementada com alocação dinâmica

Refaça o exercício anterior usando alocação dinâmica de memória.

Neste exercício, sua classe Fila deverá ter apenas um construtor e não deve conter um limite máximo de registros.